

Laboratorio 07 · Failover, mantenimiento rolling y geodistribución

Duración: 25 minutos · **Carpeta de trabajo:** entorno/

Objetivos

Vas a medir cuánto dura un failover automático visto desde una aplicación, a compararlo con un cambio de primario planificado, a aplicar el procedimiento de mantenimiento rolling y a configurar un write concern personalizado que exige confirmación en dos centros de datos.

Necesitas **dos terminales** en tu equipo, ambas en la carpeta entorno/.

Parte A · Failover no planificado

Terminal 1: entra en el toolbox y lanza la aplicación simulada:

```
docker compose exec toolbox bash
mongosh "$RS" --quiet /scripts/escritor.js
```

Terminal 2: averigua quién es el primario y "rómpelo" de golpe, como si el servidor se hubiera apagado:

```
docker compose exec toolbox bash /scripts/primario.sh
docker compose kill mongo1          # sustituye mongo1 por el primario
actual
```

Observa la terminal 1 durante 30 segundos.

1. ¿Cuántos segundos ha tardado la aplicación en poder escribir otra vez?
2. ¿Ha visto la aplicación algún error, o solo una escritura lenta? ¿Por qué?

Mira cómo lo vivieron los otros nodos:

```
docker compose logs mongo2 mongo3 | grep -iE "election|Transition to
PRIMARY|heartbeat" | tail -15
```

Recupera el nodo y observa cómo vuelve como secundario y se pone al día:

```
docker compose start mongo1
docker compose exec toolbox bash /scripts/estado.sh
```

▼ Ver solución

Lo habitual es que la aplicación esté entre 10 y 15 segundos sin poder escribir: los secundarios tardan hasta `electionTimeoutMillis` (10 s) en darse cuenta de que el primario no responde, más el tiempo de la elección y de que el driver descubra al nuevo primario. Normalmente la aplicación **no ve un error**, sino una escritura lenta: el driver espera a que aparezca un primario (hasta `serverSelectionTimeoutMS`, 30 s por defecto) y, gracias a `retryWrites=true` (activado por defecto), reintenta la escritura una vez. El nodo que vuelve entra como `SECONDARY` y recupera lo que le falta desde el oplog. Si hubiera tenido escrituras no replicadas cuando cayó, las desharía (*rollback*) y las dejaría en el directorio `rollback/`.

Parte B · Cambio de primario planificado

Con el escritor todavía en marcha en la terminal 1, conéctate al replica set en la terminal 2 y fuerza un cambio de primario ordenado:

```
docker compose exec toolbox bash /scripts/mongosh-rs.sh
```

```
> rs.stepDown(60)
```

¿Cuánto dura ahora el corte en la terminal 1? ¿Por qué es tan distinto?

▼ Ver solución

Normalmente uno o dos segundos. Con `stepDown` no hay que esperar a que venza ningún timeout: el primario deja de aceptar escrituras, espera a que algún secundario esté al día y ese secundario convoca la elección inmediatamente. Por eso **cualquier mantenimiento planificado sobre un primario empieza con `rs.stepDown()`**, nunca con un apagado directo.

Parte C · Mantenimiento rolling

Vamos a simular una ventana de parcheo del sistema operativo: hay que reiniciar los tres nodos sin que la aplicación deje de funcionar. El escritor sigue en marcha en la terminal 1. El procedimiento es siempre el mismo:

1. Secundarios primero, **de uno en uno**.
2. Después de cada reinicio, esperar a que el nodo vuelva a `SECONDARY` y sin lag.

3. El primario al final: `rs.stepDown()`, esperar a que haya nuevo primario, y reiniciar el antiguo.

En la terminal 2, identifica los secundarios y reinicia el primero:

```
docker compose exec toolbox bash /scripts/estado.sh
docker compose restart mongo3          # un secundario
```

Comprueba que vuelve a estar en `SECONDARY` y con lag 0 antes de continuar (repite el comando hasta que sea así):

```
docker compose exec toolbox bash /scripts/estado.sh
```

Repite con el otro secundario. Por último, haz `rs.stepDown()` en el primario, espera al nuevo primario y reinicia el antiguo. Escribe en tu cuaderno cuántas escrituras lentas ha visto la aplicación durante toda la operación.

Parte D · Descubrimiento de nodos

Conéctate dando **una sola** dirección en la semilla:

```
docker compose exec toolbox bash
mongosh "mongodb://admin:CursoMongo2026@mongo2:27017/?
replicaSet=rs0&authSource=admin"
```

```
> db.hello().hosts
> db.hello().primary
> db.getSiblingDB("tienda").pruebas.insertOne({ via: "semilla única" })
```

Ahora conéctate a un **secundario** en **conexión directa**. El ejemplo usa `mongo3`; si ahora mismo es el primario, cambia el nombre por el de un secundario:

```
mongosh "mongodb://admin:CursoMongo2026@mongo3:27017/?
authSource=admin&directConnection=true"
```

```
> db.getSiblingDB("tienda").pruebas.insertOne({ via: "directa" })
> db.getSiblingDB("tienda").pruebas.countDocuments()
> db.getMongo().setReadPref("secondaryPreferred")
> db.getSiblingDB("tienda").pruebas.countDocuments()
```

▼ Ver solución

Con `replicaSet=rs0`, el driver usa la semilla solo para entrar: pregunta con `hello` quiénes son los miembros y quién es el primario, abre conexiones a todos y los vigila periódicamente. Por eso basta una dirección, aunque en producción conviene poner al menos dos o tres por si la semilla está caída. Con `directConnection=true` el driver habla únicamente con ese nodo: si es secundario, la escritura falla con `NotWritablePrimary` y la lectura falla con `NotPrimaryNoSecondaryOk` hasta que permites leer de secundarios con la `read preference`.

Parte E · Geodistribución: confirmación en dos centros de datos

En el laboratorio 06 etiquetaste `mongo1` y `mongo2` con `dc: "madrid"` y `mongo3` con `dc: "barcelona"`. Comprueba que los tags siguen ahí y define un modo de `write concern` que exija confirmación en dos valores distintos del tag `dc`:

```
> rs.conf().members.map(m => ({ host: m.host, tags: m.tags })))
> cfg = rs.conf()
> cfg.settings.getLastErrorModes = { dosCPD: { dc: 2 } }
> rs.reconfig(cfg)
> db.getSiblingDB("tienda").pruebas.insertOne({ critico: true }, {
  writeConcern: { w: "dosCPD", wtimeout: 5000 } })
```

Ahora simula la caída del CPD de Barcelona:

```
docker compose stop mongo3
```

```
> db.getSiblingDB("tienda").pruebas.insertOne({ critico: true }, {
  writeConcern: { w: "dosCPD", wtimeout: 5000 } })
> db.getSiblingDB("tienda").pruebas.insertOne({ normal: true }, {
  writeConcern: { w: "majority" } })
```

Arranca de nuevo `mongo3` al terminar: `docker compose start mongo3`.

1. ¿Qué escritura falla y cuál no?
2. Con solo dos CPD, si cae el CPD de Madrid (`mongo1` y `mongo2`), ¿puede `mongo3` convertirse en primario? ¿Cómo lo resolverías?

▼ Ver solución

La escritura con `w: "dosCPD"` devuelve `writeConcernError` porque solo hay confirmación en un valor de `dc`; la de `w: "majority"` funciona porque dos de tres nodos siguen vivos. Si cae Madrid, `mongo3` es solo un voto de tres: no tiene mayoría y **no puede ser primario**; el replica set queda en solo lectura. Con dos CPD siempre habrá un lado que

no puede elegir primario por sí solo. La solución habitual es un **tercer sitio** (otro CPD, una región cloud o, como mínimo, un nodo en una tercera ubicación) para que la caída de cualquier CPD deje mayoría: por ejemplo, 2 + 2 + 1 nodos en tres ubicaciones.

Para terminar

Para el escritor con Ctrl+C. Comprueba que los tres nodos están sanos antes de irte:

```
docker compose ps
docker compose stop
```